ESSENTIAL OS

Sam & Ida

Local Search and Personal Assistant Specification

A deterministic OS knowledge engine alongside a tiered, voice-first personal assistant.

Version 1.1 | Consolidated specification
Updated 3 November 2026
Status: Working product and engineering specification
Platform reference: Essential OS; initial Ida target is Fedora KDE Plasma

CORE SEPARATION

Sam is the operating system's fast, deterministic search and retrieval engine. Ida is the user-facing personal assistant that understands requests, controls the system, converses and escalates complex work. Sam is not a small Ida, and Ida is not a conversational wrapper around Sam.

DESIGN INTENT

Move intelligence into deliberate engineering at build time, so everyday local search feels immediate without depending on a generative model at runtime.

Document control

Field	Value
Document	Essential OS: Sam & Ida - Local Search and Personal Assistant Specification
Version	1.1
Date	3 November 2026
Supersedes	Ida: Tiered Local Assistant Spec, dated 1 November 2026
Primary scope	Sam search engine, Ida assistant, their boundary and integration
Implementation stance	Design targets are provisional until benchmarked on target hardware

Change summary

- Introduces Sam as Essential OS's native, non-generative search and retrieval engine.
- Separates Sam from Ida in purpose, runtime, memory, routing, permissions and success measures.
- Defines Sam's local-computer knowledge boundary, dense registry, embeddings, deterministic handlers and caching rules.
- Adds proposed memory, latency, resource and accuracy targets, explicitly marked as unvalidated engineering budgets.
- Clarifies that Ida can call Sam through a typed interface, while ownership of external escalation remains with Ida or the Essential OS caller.
- Retains the existing Ida tiered architecture, executor policy, privacy gate, OS integration, training, testing and phased plan, with integration points clarified.

Contents

- 1. Product definition and system boundary
- 2. Shared architecture and Sam-Ida integration
- 3. Sam: deterministic OS search engine
- 4. Ida: tiered personal assistant
- 5. Shared security, privacy and operational rules
- 6. Build, evaluation and release plan
- 7. Risks, decisions and open questions
- Appendix A. Sam registry entry example
- Appendix B. Sam-to-caller contract example

1. Product definition and system boundary

1.1 Two products with different jobs

Essential OS contains two complementary systems. They may share carefully defined operating-system data services, but they do not share a single intelligence model or registry.

System	Primary job	Runtime character	Owns
Sam	Find, classify, retrieve and present information about the local computer and its contents.	Deterministic; non-generative; no reasoning loop; no conversational memory.	Global search experience, local search registry, retrieval, ranking, result formatting and operational cache.
Ida	Understand the user's request, perform system actions, converse and coordinate complex work.	Voice-first, contextual, tiered; uses deterministic paths and models where justified.	Conversation, voice interaction, command interpretation, action policy, privacy gate and escalation.

1.2 Sam - the Essential OS search assistant

Sam is the native search assistant embedded in Essential OS. Its knowledge boundary is the computer it runs on: the operating system, installed applications, settings, hardware, devices, services, local documentation, indexed files and other permitted local information. Sam answers questions and retrieves results only when they can be grounded in local indexes, registered handlers or live operating-system data.

Sam is not an LLM, chatbot, agent or general-purpose assistant. It does not reason through open-ended problems, generate original explanations, write content, maintain a conversation or build a personal profile. It recognises supported intent, retrieves the appropriate registry entry, runs a fixed handler and presents structured results. When it cannot confidently answer from its domain, it returns a typed handoff result rather than guessing.

1.3 Ida - the personal assistant

Ida is the personal assistant that the user speaks or types to. Ida handles common commands through a fast deterministic path and a small action model, then escalates requests that require open-ended generation, research, reasoning or multi-step work to an approved local or cloud backend. Ida may use Sam as a tool for local computer knowledge, search and entity resolution. Sam does not become Ida's reasoning engine.

1.4 The calculator analogy

MENTAL MODEL

Sam should feel like a highly capable FX-991-style scientific calculator: fast, reliable, domain-specific and equipped with many useful operations. It is a multi-tool for the computer, not a programmable graphing computer like an advanced TI. Ida is the more capable assistant that can decide what problem to solve, combine tools and call other systems when the problem exceeds Sam's fixed domain.

1.5 The hard boundary

Question or task type	Expected owner
Find a file, application, setting, local document or device	Sam retrieves and presents results; Ida may invoke Sam for the same lookup.
What is using my RAM? How much disk space is free? Which display is connected?	Sam reads approved live system data and returns a structured answer.
Open a found file, launch an application or navigate to a setting	Sam returns a result action; the OS UI or Ida performs the permitted action through the appropriate adapter.
Set volume, create a timer, move files, shut down, or perform another system action	Ida interprets and authorises the command; the Ida executor performs it.
Explain a concept unrelated to the local computer, write, plan or reason	Ida or the OS caller hands off to web search, a local model or a cloud model.
Current external information or research	Web search, selected by the caller; Sam does not browse.

NON-NEGOTIABLE

Sam must not expand its scope by pretending to know general knowledge. An unsupported or low-confidence query is an honest no-match or out-of-domain result, followed by a handoff controlled by the caller.

2. Shared architecture and Sam-Ida integration

2.1 High-level topology

Entry point	Primary path	Fallback / handoff
Essential OS global search	Query -> Sam -> local registry/index -> deterministic handler -> structured result -> search UI.	If unsupported, return a typed handoff to the OS search experience. The caller may offer web, local-model or cloud-model search.
Ida voice or typed input	Transcript -> Ida Tier 0 / Tier 1 -> policy-enforcing executor or Sam lookup API.	Ida router and privacy gate choose an allowed Tier 2 backend when reasoning, writing, research or complex work is needed.
Shared operating-system data	Read-only catalogues and typed OS adapters expose files, apps, hardware and current system state.	Adapters fail explicitly and report freshness / source state; callers never assume missing data is zero or success.

Sam is a peer OS service, not a fourth tier inside Ida. It can be called by the global search UI and by Ida. Ida keeps its Tier 0, Tier 1 and Tier 2 architecture. Sam does not call Ida, Hermes or a model directly.

2.2 Shared components, separate registries

- Shared low-level data services are allowed: a file catalogue, desktop-entry catalogue, settings metadata, device inventory, documentation index and read-only system-information adapters.
- Sam owns the search registry: supported search domains, query patterns, aliases, embedding vectors, ranking configuration, handler identifiers and response schemas.
- Ida owns the action registry: Act, Ask, Explain and Meta intents, slots, permissions, executor contracts, confirmations, undo behaviour and action evaluation cases.
- Do not maintain duplicate copies of the same file, app or device index. Expose one versioned data service and let each product consume it through a stable API.
- Sharing an index does not mean sharing memory, conversation state, policies or decision-making.

2.3 Ida calling Sam

When Ida receives a question about the local computer, it may call Sam rather than rebuilding local search logic inside Tier 1 or Tier 2. Ida remains responsible for the user-facing interaction: it can speak the result, ask a clarification, combine the result with conversation context or continue with a separate action. Sam returns data, not a personality or a conversational response.

Examples: Ida can ask Sam to resolve “the spreadsheet I edited yesterday”, retrieve a list of memory-heavy processes, locate a setting or count images in a permitted folder. Ida then decides what to say or do. If the user asks Ida to change a setting, Sam may locate the setting but the Ida executor owns the state-changing action.

2.4 Handoff ownership

Sam does not decide whether a query should go to the web, a local LLM or a cloud LLM. Sam has no provider policy, no model selector and no network egress. It returns a structured outcome such as `out_of_domain`, `no_confident_match`, `unsupported_operation` or `source_unavailable`. The caller owns the next step.

- When invoked from Ida, Ida's router and privacy gate select an approved backend using the reason tag and data-class rules in section 4.
- When invoked from global search, Essential OS may expose web search or an explicitly enabled local/cloud model action. The choice should be visible to the user and governed by OS privacy settings.
- A handoff must not silently transfer local file contents, paths, secrets or private metadata to an external provider. Existing consent and redaction rules remain in force.

2.5 Interfaces and process boundaries

Sam should expose a small typed local API over an appropriate local IPC mechanism. The contract should accept a query, optional search scope, caller identity/capability and bounded context required for disambiguation. It should return typed result records or a typed handoff. It must not accept arbitrary executable instructions or unrestricted shell text.

The global search UI and Ida are clients. Sam is not a privileged executor. State-changing operations remain behind the policy-enforcing execution boundary defined for Ida and the operating system.

3. Sam: deterministic OS search engine

3.1 Product principles

- Fast first. A common local query should feel immediate, with no network round-trip and no generative-model startup.
- Local-computer scope. Sam knows the computer and its permitted local information, not the outside world.
- No reasoning. Sam matches, retrieves, ranks, runs fixed handlers and formats results. It does not deliberate or construct plans.
- No conversational memory. Sam does not retain dialogue, user preferences, personal history or a profile.
- Cache, do not remember. A bounded operational cache may reuse recent results; it is disposable and has no personal meaning.
- Deterministic execution. A registry entry selects a prewritten, typed handler. User text never becomes a shell command.
- Honest boundaries. Ambiguous, unsupported, stale or out-of-domain requests produce a clarification or handoff, never a fabricated answer.
- Useful without AI branding. The product should feel more capable than a narrow pre-AI search box through breadth and careful engineering, not through a large runtime model.

3.2 Knowledge boundary and domains

Sam's domain is everything inside the computer and the computer itself, subject to user permissions and technical access controls. The initial registry should be broad enough to cover everyday computer discovery and support tasks, then grow from measured query demand.

Domain	What Sam can retrieve or answer	Typical examples
Filesystem and local content	Names, types, locations, metadata, recent items, indexed contents and permitted relationships.	Find a PDF; count images; locate the latest modified spreadsheet; search a local project.
Applications	Installed applications, desktop entries, versions, categories, launch targets and application help available locally.	Find the screenshot tool; show installed browsers; locate the application that edits RAW photos.
Settings and OS navigation	Settings taxonomy, supported setting values, setting-page identifiers and navigation targets.	Where is night light? Open display settings; find keyboard accessibility settings.
Hardware and devices	Detected CPU, memory, storage, battery, displays, audio devices, Bluetooth devices and supported peripherals.	What GPU do I have? Which audio output is active? Is the second display detected?
Live system state	Approved live metrics and process/service information exposed by OS interfaces.	What is using RAM? How much disk space is free? Is Wi-Fi enabled?
Networking	Local interfaces, link status, saved network names only where explicitly permitted, connection state and diagnostics.	Which network am I connected to? Is Bluetooth on? Why is this interface disconnected?

Domain	What Sam can retrieve or answer	Typical examples
Media and codecs	Local media players, libraries, playback state, supported codec information and media-file metadata.	What is playing? Which application can open this video? Find my music files.
Local documentation	Bundled help, manuals, installed man pages, local knowledge base and documentation shipped with applications.	How do I change a setting in this installed app? Find the guide for the backup utility.
Essential OS itself	Features, services, components, releases, update state, bundled applications, known troubleshooting steps and OS documentation.	What does Essential OS include? Check update status; find the network panel documentation.

The domain is intentionally broader than settings search. It includes the local machine, local data and installed software. It does not include arbitrary facts about the world, current news, general research, open-ended writing, or questions whose answer depends on external live information.

3.3 Out-of-domain requests

The following are not Sam responsibilities: general factual questions unrelated to the computer; current events; broad research; web content discovery; writing or rewriting; open-ended explanation; complex diagnosis that requires causal reasoning beyond registered checks; multi-step planning; autonomous agent tasks; and general-purpose conversation.

If a query mixes local and external requirements, Sam may return the local portion if the intent is explicitly supported, but it must mark the external portion as a handoff. It must not pretend a local index is a substitute for current web information.

3.4 Query pipeline

1. Input normalisation: trim, tokenise, normalise common contractions, punctuation, units and known OS vocabulary. Preserve the original query for display and diagnostics only where policy permits.
2. Fast lexical path: match exact phrases, aliases, keywords, synonyms, domain names and known slot patterns. This path should resolve common queries without running semantic inference.
3. Semantic retrieval: when lexical retrieval is insufficient, encode the query with a compact local embedding encoder and compare it with precomputed registry vectors. Embedding similarity is a retrieval signal, not reasoning.
4. Candidate ranking: combine lexical score, semantic similarity, domain prior, slot compatibility, freshness and source availability. Apply calibrated confidence thresholds and ambiguity margins.
5. Dispatch: select one registered handler, request one concise clarification, return results, or issue a typed handoff. Never infer an unregistered operation.
6. Execution: invoke a predefined read-only handler or an explicitly allowed result action. Validate all parameters and permissions before accessing the source.
7. Response: return structured results with provenance, freshness and supported UI actions. Use deterministic templates for short summaries.
8. Cache: store an eligible result or query plan in the bounded cache and invalidate it when its dependencies change.

3.5 Registry design

The registry is the central source of truth for Sam's supported knowledge. Each canonical capability should be represented once and may have many phrasings. Registry data is versioned, reviewable, testable and compiled into runtime-friendly artifacts.

Registry field	Purpose
id / version	Stable intent identifier and schema version.
domain	Filesystem, applications, system, hardware, settings, documentation, Essential OS, or another approved domain.
canonical_intent	The single operation or retrieval intent represented by this entry.
description	Human-readable definition and explicit boundaries.
phrases / aliases	Seed queries, paraphrases, synonyms, colloquial wording, abbreviations and common misspellings.
keywords / lexical rules	Fast exact and token-based matching features.
embedding_text / vectors	Canonical and paraphrase text used to produce precomputed registry embeddings.
slots	Typed parameters such as path, file kind, application, device, time range or result count.
source	Authoritative local index, OS interface, settings catalogue or documentation collection.
handler	Stable identifier for a prewritten, version-controlled handler.
permissions	Read scope, protected paths, required capabilities and whether a result action is allowed.
response_schema / template	Typed output contract and concise deterministic presentation.
fallback / escalation	Clarification rule, no-match threshold and caller handoff reason.
eval_cases	Positive, ambiguous, negative, out-of-domain and adversarial test cases.
cache_policy	Eligibility, TTL, dependency tags and invalidation triggers.

3.6 Dense registry authoring

During Essential OS development, a highly capable model may be used as a registry-authoring tool. Examples include a frontier coding or reasoning model available at build time. It can draft domain taxonomies, canonical intents, paraphrases, synonyms, slot definitions, test cases, response templates and candidate handler mappings. It may also help identify gaps and confusing overlaps.

> **BUILD-TIME INTELLIGENCE, RUNTIME SIMPLICITY**
>
> The authoring model is a development dependency, not a Sam runtime dependency. It is not shipped with Sam, is not called to answer user queries and is not part of the installed memory footprint. Every generated registry entry must be reviewed, validated against a real local source, tested and versioned before release.

The registry should be dense enough that users can phrase supported searches naturally rather than memorising commands. Density comes from broad domain coverage and high-quality aliases, not from creating a unique script for every sentence.

3.7 Embeddings and retrieval

Registry embeddings are precomputed during the build pipeline and stored with the compiled registry. At runtime, Sam needs a way to represent the new user query in the same vector space. The preferred design is a small, dedicated sentence-embedding encoder, distinct from an LLM, loaded only when the lexical path cannot confidently resolve the query.

- The embedding encoder generates vectors; it does not generate answers, perform chain-of-thought reasoning, plan actions or retain conversational state.
- Use a fast lexical path first. Run embedding inference only for uncertain or paraphrased queries where semantic retrieval adds measurable value.
- Use a flat vector index or another lightweight structure while the registry is modest. A heavyweight vector database is not a default requirement.
- Store vectors in a compact numeric format and keep domain partitions so the search space can be narrowed before comparison.

- The encoder must be benchmarked for quality, cold-start time, peak RAM and CPU cost on low-end x86-64 hardware without an NPU.
- If the encoder is unavailable, slow or over budget, fall back to lexical retrieval and return a safe no-match where confidence is inadequate.

A dedicated embedding encoder is still a small machine-learning component in the technical sense, but it is not a large language model and it does not make Sam a reasoning system. A purely lexical configuration should remain possible for constrained hardware or debugging.

3.8 Prewritten handlers and scripts

Each canonical intent maps to a reusable, version-controlled handler. A handler reads from an approved source, validates typed inputs, returns a typed result and reports errors explicitly. The handler must not assemble a shell command from user text.

- Many phrasings should map to one canonical intent and one reusable handler.
- Use fixed argument arrays, structured APIs and typed OS interfaces. Never pass raw query text to a shell.
- Prefer read-only discovery. Opening a file, launching an app or opening a settings page should be represented as a constrained result action, not arbitrary code execution.
- State-changing actions such as changing volume, moving files, deleting files, rebooting or shutting down belong to Ida's policy-controlled executor or another explicit OS action service.
- A handler must state its source, expected latency, permissions, timeout, cacheability and invalidation triggers.

3.9 Result model and presentation

Sam returns structured results rather than generated prose. A result may include a title, concise deterministic description, entity type, source, location, version or state, last-updated time, confidence band and a set of allowed actions. Lists and comparisons remain visual. Spoken use through Ida should be summarised by Ida, not by a Sam speech engine.

Templates should be concise and literal. A result must not claim more than its source supports. For volatile values such as temperature, battery, RAM and connectivity, show the time of measurement or refresh the value on demand.

3.10 Caching without memory

Sam may cache frequently used queries and eligible results to make repeated searches faster. This is operational caching, not personal memory. It stores no user identity, relationship, preference, conversational context or long-term history.

Allowed cache behaviour	Disallowed memory behaviour
Bounded in-memory LRU cache of recent query plans or safe results.	Persistent conversational history or a transcript archive.
Short TTL for dynamic system data; invalidate on relevant state changes.	Remembering what a user asked yesterday or inferring personal preferences.
Cache stable metadata and registry lookup results when their source version is unchanged.	Building a user profile, personal timeline or behavioural model.
Cache key includes normalised query, scope, permission context and relevant source/version identifiers.	Reusing a result across users, scopes or permission levels.
Cache is disposable and can be cleared at any time.	Treating the cache as an authoritative source of truth.

Dynamic data must have short TTLs or event-driven invalidation. File and application results must be invalidated when the underlying index changes. Sensitive file contents, secrets and protected metadata should

not enter the result cache by default. Query-frequency statistics should not become a permanent per-user history; aggregate development telemetry, if collected, must be opt-in, minimised and local.

3.11 Indexing and freshness

- Maintain local catalogues for supported domains using the OS's available change notifications and incremental refresh mechanisms.
- Index only permitted roots and sources. Respect ignore lists, denied paths, permissions and application privacy boundaries.
- Separate metadata indexing from full-content indexing. Full-content search is optional, explicitly scoped and resource-budgeted.
- Never index secrets such as tokens, private keys, password stores or credential databases.
- Show freshness when results depend on an index that may be stale. Offer a refresh or live query rather than silently presenting old data as current.
- On first boot or after a major index rebuild, perform bounded background indexing with visible status and a pause mechanism. Sam's query path must remain responsive.

3.12 Sam resource and latency targets

Sam is designed to run comfortably on a low-resource computer, including a system with 4 GB of physical RAM. The following are initial engineering budgets, not measured guarantees. Phase S0 must benchmark the complete pipeline on the minimum target machine before the values become release gates.

Component / metric	Initial budget or target	Notes
Core search process	5-25 MB	Approximate design allowance; implementation-dependent.
Registry and lexical indexes	10-100 MB	Depends on registry density, index format and enabled domains.
Embedding encoder	50-200 MB	Optional and preferably loaded on demand; benchmark actual resident and peak memory.
Bounded cache	16-64 MB	Hard cap; no unbounded growth.
Temporary working memory	10-50 MB	Query vectors, result buffers and short-lived data.
Idle resident memory	Under 100 MB	Target with optional encoder unloaded.
Typical query peak	Under 250 MB	Target with normal search path; validate with encoder path.
Hard process ceiling	Under 512 MB	Initial maximum budget; failure should degrade to lexical search rather than grow without bound.
Supported system memory	4 GB RAM	Minimum product target for Sam; verify with OS baseline and concurrent workload.
Exact / lexical match	Under 10 ms	Internal retrieval target, warm process, excluding source access and UI rendering.
Cache hit	Under 5 ms	Internal result lookup target.
Semantic retrieval	Under 50 ms	p95 target for query encoding plus ranking on reference low-end hardware.
Local search result	Under 100 ms	p95 target for common metadata and registry queries, excluding slow external devices.
Live system information	Under 100 ms	p95 target for simple metrics when source APIs respond normally.
Idle CPU	Near zero	No continuous model inference; background indexing is separately throttled.

These allocations are planning estimates and must not be treated as measured requirements. A 4 GB target means Sam should remain useful on such a system under realistic OS load; it does not mean every index and optional encoder must remain permanently resident.

3.13 Sam success criteria

Metric	Proposed acceptance target	How to measure
In-scope retrieval	At least 98% top-1 on clean supported queries and 95% or better on varied paraphrases.	Frozen query set with gold intent and expected result source.
Out-of-domain handling	At least 95% of out-of-domain queries correctly handed off.	Mixed in-scope / out-of-scope evaluation set.
Confident wrong dispatch	1% or less.	Adversarial, ambiguous and near-neighbour tests.
Latency	Meet the p95 targets in section 3.12.	Cold/warm latency harness, under normal and competing load.
Memory	Typical peak under 250 MB; hard ceiling under 512 MB; supported on 4 GB systems.	RSS and peak memory tracking over long runs and large indexes.
Offline operation	All local domains and handlers continue to work without network access.	Network-disabled integration tests.
No generative dependency	No LLM inference, no cloud request and no hidden model fallback in Sam.	Process, dependency and outbound traffic audit.
No personal memory	No conversation store, user profile or persistent query history.	Storage inspection and behavioural tests.
User-perceived speed	Users consistently describe search as immediate and are surprised it uses no LLM.	Task-based usability sessions and post-task interviews.

The product-level success moment is intentional: a user phrases a local-computer question naturally, receives a useful answer almost immediately, asks what AI is running, and learns that the result came from a carefully engineered registry and scripts rather than an LLM.

3.14 Sam failure behaviour

- Low confidence: return no confident match and offer a relevant domain or caller handoff.
- Ambiguous entity: ask one short clarification and show a small numbered result list where useful.
- No source data: report which source is unavailable; never substitute a guessed value.
- Stale index: disclose staleness and refresh the relevant domain within the resource budget.
- Handler timeout or adapter failure: stop, return a structured error and avoid partial claims.
- Embedding encoder unavailable: use lexical path; do not block the search UI waiting for a model restart.
- Out-of-domain query: return a typed handoff without attempting to answer from unrelated local material.

4. Ida: tiered personal assistant

4.1 Summary and goals

Ida is a voice-first assistant for the Fedora KDE laptop and, later, Essential OS. It handles common requests with a fast closed-intent action system and escalates the long tail to a local or cloud model through a privacy gate. Pre-AI Siri felt instant because it was narrow: a fixed menu of intents, deterministic execution and templated replies. Ida keeps that fast common path while retaining an escape hatch for complex requests.

Initial reference machine: HP ProBook 635 Aero G8 running Fedora KDE Plasma, with no NPU. All local inference must work on CPU or iGPU. Phase 0 measures actual RAM headroom and revises model limits and performance budgets where necessary.

Goal	Target
Coverage	Tier 0 and Tier 1 resolve at least 80% of real logged utterances without escalation.
Speed	Wake word to visible or audible feedback at 150 ms or less; end of speech to execution p95 at 600 ms or less for Tier 0 and 1,000 ms or less for Tier 1.
Accuracy	At least 95% exact match on intent and slots for in-scope utterances; confident-but-wrong actions at 1% or less; at least 95% of out-of-scope utterances escalate.
Safety	Zero actions outside allowed filesystem scope in adversarial tests. Destructive actions are confirmed and undoable where the OS allows.
Privacy	Zero bytes of private-class data sent to a cloud model without recorded consent.
Offline	Tiers 0 and 1, all Act and Ask intents, and a local Tier 2 model work without a network.
Extensibility	A new intent ships as a registry entry plus evaluation cases; no model retraining is required by default.

4.2 Non-goals

- Not a standalone general chatbot. Open-ended conversation, writing and reasoning belong to Tier 2.
- Not a replacement for Hermes. Hermes remains the agent loop for multi-step work; Ida is the fast front tier and decides when to call it.
- No new speech models. Wake word, ASR and TTS use existing open-source components; Ida does not train or fine-tune speech systems.
- No personal knowledge in model weights. File names, app names and device state come from live indexes and runtime state.
- No permission logic in the model. Path limits, confirmations and policy are enforced by code in the executor.
- No raw shell for Tier 1. It emits only typed calls from the registry.
- No pixel-level GUI automation in v1. Integrations use D-Bus, CLI tools and app APIs; screen-driving, if added, is a Tier 2 capability.
- No proactive behaviour. Ida acts only when asked; no unprompted suggestions or background actions.
- No learned router in v1. Routing is rule-based and revisited only if logs justify it.
- No multi-device, multi-user, Windows or GNOME support in v1. Fedora KDE is the first target; OS access remains behind adapters for later ports.
- No plugin marketplace. Intents are added by the owner through the registry.
- English only in v1. Other languages are later work.
- No raw audio retention by default. Audio is discarded after transcription unless capture is enabled for debugging.
- No 100% coverage target. The long tail escalates by design; defer intents that cover less than 0.5% of requests unless they are strategically important.

4.3 Ida architecture

Ida has three processing tiers and one policy-enforcing executor. Every action, whether proposed by Tier 0, Tier 1 or a Tier 2 agent, passes through the same executor. Sam is an adjacent local search service that Ida may call; it is not an Ida tier.

Component	Responsibility	Network access
Audio service	Wake word, voice activity detection, ASR and TTS.	No network required in the local path.
Ida Core	Tier 0, Tier 1, router, conversational state block and entity resolvers.	No network.
Executor	OS adapters, permission checks, confirmations, verification and audit.	No general network egress.
Tier 2 gateway	Hermes and approved local/cloud model backends.	Only Ida process with network egress.
Sam service	Separate deterministic search, registry, local indexes and query cache.	No network egress.

4.4 Ida tool registry

Every Ida capability is one registry entry. The registry is the single source of truth for the action vocabulary, executor contract, help text and evaluation cases. Adding an intent does not require retraining by default.

Class	Side effects	Confirmation	Reply
Act	May change OS state.	Destructive operations only by default; offer undo where available.	Short spoken confirmation.
Ask	No side effects.	Never.	Templated answer from live data or Sam.
Explain	No side effects.	Never.	Retrieved snippet or escalation.
Meta	Control Ida interaction.	Depends on operation.	Brief state response.

Entry fields: name, class, description, examples (at least 10 seed utterances), slots, executor, permissions, response template, modality (speak, show or both), undo, timeout_ms and offline_ok.

Slots and entity resolution

Supported slot types include enum, number with unit, duration, time, path_ref, app_ref, device_ref and player_ref. The model emits the raw spoken string for entity slots. A resolver maps it to ranked candidates using fuzzy and embedding matching against a live index. Sam may provide the shared local catalogues or a typed lookup endpoint; Ida retains the policy and clarification responsibility.

Auto-pick only when the top score is at least 0.85 and leads the runner-up by at least 0.15. Otherwise ask one short disambiguation question. The user-facing reply should state the resolved entity so that a wrong selection is immediately visible.

Starter intent set

Domain	Intents	Class
Audio	set_volume, change_volume, mute, unmute, set_output_device	Act
Display	set_brightness, change_brightness, night_light	Act
Focus	focus_mode on/off with optional duration	Act
Clock	set_timer, cancel_timer, set_alarm, cancel_alarm	Act
Clock	list_timers, current_time, current_date	Ask
Media	play, pause, next, previous, seek, player_volume	Act
Apps and windows	open_app, close_app, focus_window, move_window_to_desktop	Act
Files	open_file, reveal_file, create_folder, move_file, trash_file	Act; move/trash destructive
File lookup	find_file, count_files, file_info, recent_files	Ask; local lookup may delegate to Sam

Domain	Intents	Class
System info	ram, cpu_load, disk_space, battery, uptime, os_version	Ask; may use Sam live-data handlers
Network	wifi on/off, bluetooth on/off, wifi_status	Act and Ask
Identity	whoami, hostname	Ask
Power	lock_screen, suspend, reboot, shutdown	Act; shutdown confirms
Capture	take_screenshot	Act
Help	what_can_you_do, how_do_i	Explain
Meta	undo, repeat, cancel, escalate	Meta

Growth rule: add an Ida intent only when escalation logs show it at 0.5% or more of requests, or when it is required for safety, accessibility or a core product capability. Each intent requires a schema, executor, evaluation cases and a confusion check.

4.5 Tier 0 and Tier 1

Tier 0 handles rigid phrasing with no model. Tier 1 is a small model that turns an utterance plus live state into one typed call, one clarifying question or an escalation.

Speech front end

Wake word (openWakeWord or similar), voice activity detection, then local ASR. Phase 0 benchmarks whisper.cpp, faster-whisper (small and distilled variants, int8) and Moonshine on the target laptop. Where supported, bias recognition with vocabulary from the Ida registry and live index, including app and folder names. V1 uses the top transcript only.

Tier 0 grammar fast path

- Compile patterns from the Ida registry seed examples and slot types so the grammar cannot drift from the registry.
- Match exact or near-exact phrasing with parsed numbers, durations and times (for example, “volume 30” or “timer 10 minutes”).
- A non-match falls through to Tier 1 untouched. Runtime target is under 10 ms.

Tier 1 small action model

Element	Specification
Model	0.5B to 1.5B parameters, 4-bit quantised, served by llama.cpp and resident from login if the Phase 0 RAM budget allows.
Baseline	Start with an untrained instruct model and few-shot examples. LoRA fine-tuning only if baseline misses accuracy targets.
Prompt	Fixed system header (KV-cached), top 6 tool specs retrieved by embedding similarity, live state block, last 3 turns and utterance; target under 600 tokens.
State block	Volume and mute, brightness, focus mode, active window, media player/status/track, running timers, local time/date and last entity Ida referred to.
Output	One grammar-constrained JSON object (GBNF generated from shortlisted tools). Reserved outputs: escalate with a reason tag, or clarify with a question key.
Compound requests	Up to 3 independent Act calls in one utterance. Dependent or longer chains escalate.
Entity slots	Emit raw spoken strings; a resolver maps them to real files, apps or devices.

Escalation signals

1. The model emits `escalate`.
2. The log-probability margin between the top two intents is below calibrated threshold tau.
3. The retrieval shortlist's best similarity is below the floor; no registry entry resembles the request.

4. A slot resolver fails after one disambiguation attempt.
5. Tier 1 exceeds 1.5 seconds; it aborts and escalates with an acknowledgement.

Thresholds are calibrated on a held-out development set so that confident-but-wrong actions remain at 1% or less. A wrong confident action is more costly than a slow correct one.

4.6 Router and privacy gate

The privacy gate decides what may leave the machine using rules, not model judgment. Ida attaches a reason tag when it escalates; the router selects the backend only from the options permitted by the gate.

Data class	Examples	Cloud policy
Public	General questions with no personal context.	Allowed.
Metadata	File names, paths, app lists and device state.	Only with names stripped or after per-class consent.
Content	File contents, notes, photos, email and calendar.	Local only. Cloud only on explicit per-request override.
Secrets	Keys, tokens and passwords.	Never sent to any model; redact before any prompt or log.

Every piece of Tier 2 context carries a class tag assigned by resolvers and context assembly, not by the model. The gate strips or drops anything above the allowed class for the selected backend. The same rule applies to tool results flowing back into an agent loop.

Reason tag	Example	Route
private_content	“Summarise this PDF in my Documents.”	Local mid.
simple_gen	“Rewrite this sentence”; “What does this error mean?”	Local mid, or cloud fast if no private context.
needs_web	“What is the latest on...”	Cloud fast / web search.
multi_step	“Reorganise my Downloads and tell me what you did.”	Hermes with cloud strong, with the gate applied per tool result.
hard_reasoning / code	Debugging, planning or complex code work.	Cloud strong, subject to privacy gate.
unknown	No clear reason.	Cloud fast only if the gate permits; otherwise local mid or a request for clarification.

Backends and fallback

Local mid is a 3B to 8B model at 4-bit, loaded on demand; the size cap follows measured RAM headroom. Cloud fast is a low-cost model for routine generation. Cloud strong is reserved for multi-step and hard-reasoning work. Backends use an OpenAI-compatible API so provider changes are primarily a base URL and model name. If Hermes cannot switch provider per call, use a thin proxy such as a LiteLLM-style gateway.

1. Offline: requests permitted by the gate go to local mid. Ida states that it is offline and answers may be rougher.
2. Cloud timeout: if there is no first token in 8 seconds or an error occurs, retry once, then try the next cheaper cloud model, then local mid if permitted.
3. Fallback never widens privacy exposure. If the only remaining option would send a higher data class out, stop and explain why.
4. If local mid is unavailable, state that clearly rather than failing silently.

The first time a data class would leave the machine for a given backend, Ida asks once and stores the answer (class, backend, timestamp) in settings. The owner can revoke consent at any time. “Ask the big model” is the per-request override for Content. No cascades in v1: routing happens up front using the reason tag. Each escalation logs reason tag, backend, latency and whether the owner re-asked or corrected.

4.7 OS integration layer

Every OS capability is exposed through a typed adapter wrapping one OS mechanism. Executors stay small, mockable and replaceable. Phase 1 verifies each mechanism on the real machine and records a fallback.

Capability	Mechanism	Notes
Volume, mute, output device	wpctl (PipeWire / WirePlumber)	Cap at 100% unless owner says “boost”.
Brightness	PowerDevil over D-Bus; brightnessctl fallback	Probe at startup and remember working mechanism.
Night light	KWin D-Bus	Verify on target Plasma version.
Focus mode	KDE Do Not Disturb via notification inhibition	Timed focus uses Ida timer to switch it off.
Media	MPRIS over D-Bus (playerctl or direct)	Pick the playing player, else last active.
Timers / alarms	systemd transient user timers; state in Ida SQLite	Re-arm from SQLite at login; test wake from suspend.
Apps	Desktop entries via Gio and kioclient	Fuzzy match name and keywords; catalogue may be shared with Sam.
Windows	KWin scripting over D-Bus	Wayland-safe; no arbitrary input synthesis.
File index	Shared local file catalogue / Sam index API	One index owner; Ida uses typed lookup and resolvers.
Open / reveal / trash	xdg-open, kioclient, gio trash	Trash keeps deletion undoable; executor enforces scope.
System info	/proc, statvfs, UPower D-Bus	RAM, CPU load, disk, battery, uptime; Sam may expose read-only query handlers.
Network / Bluetooth	NetworkManager and BlueZ over D-Bus	Separate read status from write actions.
Power	logind over D-Bus	Lock needs no confirmation; suspend, reboot and shutdown do.
Screenshot	Spectacle CLI	Fixed arguments and bounded path.
Notifications	org.freedesktop.Notifications	Used for on-screen results.
Speech output	Piper TTS or similar, local	Streams text as it arrives.

Adapter contract: each call has a timeout and returns `ok`, `data` or `error`. Adapters implement an OSAdapter interface with mocks for tests. After every Act, the executor reads the state back and confirms success only if the read matches. Failure produces an honest failure message and a log entry.

4.8 Security and permissions

- A user-owned policy manifest lists allowed read roots, allowed write roots, denied paths (for example ~/.ssh, ~/.gnupg, browser profiles and /etc), per-tool permissions and confirmation rules. Ida runtime cannot write to the policy manifest.
- Resolve paths with realpath first (including symlinks and ..), then check allow and deny lists; deny wins. Re-check at execution time with O_NOFOLLOW and file descriptors where possible to close check-then-use gaps.
- Ida Core (Tier 0, Tier 1 and router) has no network and writes only to its own state directory. Executor runs as a separate restricted process with systemd hardening (NoNewPrivileges, PrivateTmp, read-only home and explicit writable roots). Only Tier 2 gateway has network egress.
- Executors use fixed argument arrays, never shell strings assembled from input. Validate slot values and ranges before use.
- Destructive confirmation names the exact target. Deletion goes to Trash only; permanent delete does not exist in v1. Operations touching more than five files always confirm.
- Voice alone cannot confirm destructive actions or power-off by default. A keypress or click is required to prevent wake-word false triggers or ambient audio from approving them.
- Tier 1 sees only the owner's transcribed utterance and trusted state, never file contents or web text. Tier 2 treats retrieved content as data; every Tier 2 tool call uses the same executor policy and fresh confirmation for destructive operations.
- Redact key and token patterns from transcripts and logs. Audio is discarded after transcription. Logs stay local with 30-day default retention.

- Append every executed call to a local audit log with time, transcript, resolved arguments, result and verification outcome.
- Hash-check model files and registries at startup. No automatic model downloads; updates require owner action.

4.9 Ida UX specification

Ida must feel immediate and honest. It always shows it heard, never guesses silently and never leaves the owner waiting in silence.

Moment	Target	Feedback
Wake word detected	150 ms or less	Earcon and small listening indicator.
While speaking	Live	Transcript appears as recognised.
Tier 0 / 1 action executed	Within latency budgets	Short spoken confirmation and visible state change.
Escalation starts	1 s or less	Short acknowledgement (“On it”) and thinking state with Local or Cloud badge.
Cloud first token	About 3 s	Text and speech stream as it arrives.
Failure	Immediate	Plain statement of what failed and one next step.

- Match output to channel: short facts are spoken; lists and search results appear on screen. Voice reads no more than two items.
- Say what was done and name resolved targets (for example, “Opening budget-2026 in Documents”).
- Disambiguate once and show up to three numbered options. If the top candidate clears the resolver threshold, act and state what was selected.
- Undo is a verb: “Undo that” reverses any of the last five Act calls where the OS permits (restore volume, cancel timer, restore from Trash).
- Show the transcript. “No, I meant...” reruns the correction against the previous utterance and logs the correction.
- No dead ends: an unrecognised request escalates rather than returning only “Sorry, I did not get that”.
- Be transparent about the brain: Tier 2 answers show Local or Cloud; offline state is stated plainly.
- Stay interruptible: “stop” or a keypress halts speech immediately.
- Confirmation prompts are short and specific. No response within eight seconds cancels the action.
- Silent mode switches replies to on-screen only, for meetings and shared spaces.
- Typed input uses a hotkey and feeds the same pipeline as voice; it is also the test harness input.
- “What can you do?” lists domains and examples generated from the Ida registry.
- Tone: operational responses are brief and plain. Conversational Tier 2 responses may be more natural, but must remain honest and intellectually independent.

4.10 Ida data and training plan

1. Seed utterances: at least 10 owner-written examples per intent.
2. Evaluation set: 300 to 500 utterances with gold calls, split 50/50 into development and frozen test sets. The test half is never used for training, prompt tuning or threshold calibration.
3. Synthetic training data: a strong model generates 200 to 500 paraphrases per intent across registers and slang. Pair these with state-block variants, escalation negatives and ASR noise. Use TTS-to-ASR round trips to create realistic recognition errors. Deduplicate against evaluation data by n-gram and embedding similarity.
4. Fine-tune only if necessary: LoRA or QLoRA on a 0.5B to 1.5B model with 3,000 to 10,000 examples, targeting constrained JSON output. The laptop has no discrete training GPU, so use rented compute and synthetic, non-private data only. Return weights to the laptop.

5. Calibrate thresholds on the development half and report final results on the frozen test half.
6. Promote a model or registry version only when it passes G3 on the frozen test set and does not regress against the current version. Store registry version, model version and evaluation scores together.

4.11 Ida evaluation-set composition

Category	Share	Expected behaviour
In-scope, clean phrasing	25%	Correct call.
In-scope, paraphrased	20%	Correct call.
ASR noise (homophones, dropped words, “for” / “four”)	10%	Correct call.
Disfluent speech	5%	Correct call.
Needs state (“turn it up”, “pause that”)	10%	Correct call using state.
Ambiguous	5%	Clarify.
Compound	5%	Up to three independent calls, or escalate.
Out of scope	15%	Escalate.
Adversarial	5%	Refuse or clamp.

4.12 Ida phased plan

Phase	Deliverables	Exit gate
0. Measure and decide	Benchmark ASR and 0.5B / 1.5B / 3B models on the laptop; measure RAM headroom; confirm OS mechanisms; test Hermes per-call model switching and tool policy.	Budgets recorded, ASR and Tier 1 models chosen, RAM budget set, OS mechanisms verified or fallback assigned, Hermes path decided.
1. Walking skeleton	Typed hotkey input, Tier 0 grammar, registry loader, policy engine, path checks, executor, verify-after-act, audit log, audio / brightness / media / timer intents.	Unit and path-policy suites pass; every Act verifies; owner uses text tool daily.
2. Voice and Tier 1 baseline	Wake word, VAD, ASR, TTS, retrieval shortlist, untrained Tier 1 with constrained output, state block, 300+ utterance evaluation set and latency harness.	Latency targets met on typed and voice paths; baseline scores recorded; fine-tuning decision made.
3. Full registry	About 40 intents including Ask and file lookup, resolvers, shared file catalogue, disambiguation and confirmation UX, optional LoRA and think-aloud sessions.	G3 and G4 pass on frozen test set and safety suite; UX task success at least 90%.
4. Router and Tier 2	Privacy gate, data classes, consent store, reason tags, routing table, local-mid backend, cloud backends through Hermes or proxy, fallback order and offline mode.	Canary suite shows zero leaks; fault-injection passes; all Act and Ask intents pass offline.
5. Daily use	Four weeks as daily driver; weekly log review; add intents using 0.5% rule; threshold recalibration.	G1 measured on two weeks of real logs; confident-wrong at 1% or less; diary shows Ida preferred for common tasks.

Each phase ends at a measured gate. A safe stop at any gate leaves a working system. If a gate fails twice, apply the kill and pivot criteria before spending more time.

5. Shared security, privacy and operational rules

5.1 Product boundary is also a security boundary

- Sam is read-oriented and has no network egress. It queries only approved local indexes and OS interfaces. It does not get privileged write access because it is part of the search UI.
- Ida Core is offline-capable and has no network egress. The Tier 2 gateway is the only Ida process allowed to reach local or cloud model endpoints and the web.
- The executor is the single policy choke point for state-changing operations. No model, registry entry, search result or caller can bypass it.
- A result action returned by Sam is a proposal represented by a constrained action identifier. The caller must validate it and invoke the appropriate OS capability; no arbitrary callback or shell text is permitted.
- Local content returned by Sam remains local unless the user explicitly authorises a separate Ida or OS handoff. The privacy gate applies to both first prompts and tool results.

5.2 Index and content protection

The shared catalogue must respect per-user filesystem permissions, allowed roots, denied paths, symbolic links and application privacy boundaries. Indexing must never create a new route around the executor or the operating system's access controls. Secrets are excluded from indexing and caching. Content-class data is local by default.

5.3 Prompt injection and untrusted text

Sam is deterministic and does not follow instructions found inside indexed documents. Indexed file contents, application help and local documentation are data, not commands. Ida Tier 1 must not receive untrusted content. Tier 2 treats retrieved content as untrusted data and remains subject to the same executor policy.

5.4 Logging and retention

System	Default records	Retention rule
Sam	Operational errors, handler timing and index health. No transcript or personal query history by default.	No persistent per-user search history. Optional diagnostics are local, minimised and explicitly enabled.
Ida	Audit record of executed actions, resolved arguments, result, verification and corrections.	Local 30-day default retention; redaction for secrets. Audio discarded after transcription.
Tier 2 gateway	Backend, reason tag, latency, consent and failure details.	Local minimised operational logs; no private content unless explicitly required and authorised.

5.5 Updates and integrity

- Sam registry, embedding vectors, handlers and model files (if an embedding encoder is used) are versioned as one compatible release bundle.
- Ida registry, Tier 1 model version and evaluation scores are promoted together.
- Hash-check installed bundles at startup. Do not download or replace model weights, handlers or registry content silently.
- Search index schema migrations must be reversible or rebuildable from the approved local sources.

6. Build, evaluation and release plan

6.1 Sam implementation phases

Phase	Work	Exit gate
S0. Scope and measure	Inventory local domains, data sources and OS interfaces. Benchmark process overhead, 4 GB memory environment, indexing and cold/warm query latency. Decide whether semantic encoder is necessary.	Documented resource baseline; domain source map; no critical dependency on a large model or network.
S1. Deterministic core	Implement query normalisation, domain registry, lexical match, confidence thresholds, typed result schema and handoff outcomes.	Unit tests pass; unknown and ambiguous queries fail safely; top clean-query precision meets initial target.
S2. Registry authoring pipeline	Use a build-time advanced model to draft and expand registry data. Review, deduplicate, validate handlers and generate frozen evaluation sets.	Registry is versioned, reproducible and reviewable; generated entries cannot ship without tests.
S3. Semantic retrieval	Add precomputed registry embeddings and a compact query encoder loaded on demand. Tune hybrid lexical / semantic ranking.	Semantic path improves paraphrase recall enough to justify its latency and RAM cost; lexical-only fallback remains usable.
S4. Sources and handlers	Connect file, app, settings, hardware, system, media, networking and local documentation sources. Add bounded incremental indexing and typed handlers.	Source freshness, access control, timeouts and error reporting pass integration tests.
S5. Search UI	Integrate with Essential OS global search, result templates, result actions, disambiguation and handoff UX.	Users can complete common discovery tasks without memorising phrases; search feels immediate.
S6. Ida integration	Expose Sam through a typed local API. Route Ida local-information lookups to Sam where appropriate. Preserve Ida executor and privacy policy.	No duplicate indexes; Ida can use Sam for local lookup without giving Sam action authority or model-routing responsibility.
S7. Low-resource and release validation	Test on 4 GB RAM systems, offline, under index load, with encoder cold start, failure injection and long-running cache churn.	Memory, latency, accuracy, offline, privacy and no-memory acceptance gates pass.

6.2 Sam registry build pipeline

1. Define domains and trusted local source contracts before generating registry entries.
2. Ask a capable build-time model to propose canonical intents, phrase variants, synonyms, slot schemas, handler mappings, response templates and adversarial examples.
3. Deduplicate overlapping entries and check nearest-neighbour confusion across related intents.
4. Require a human-reviewed source mapping and implementation for every executable handler.
5. Generate embeddings for canonical entries and paraphrases. Store model/version metadata with the vectors.
6. Build a frozen evaluation set with clean queries, paraphrases, ambiguous queries, unsupported requests, out-of-domain queries and adversarial strings.
7. Run the full evaluation and publish registry, handlers, index schema, embedding encoder version and metrics as one release artifact.
8. Review production-like misses and corrections. Add registry coverage only when evidence shows a real gap; do not chase unlimited phrase variants.

6.3 Shared test strategy

Layer	Sam coverage	Ida coverage	Pass condition
Unit	Normalisation, registry parsing, lexical rules, embedding lookup, ranking, cache keys, templates.	Grammar parsing, slots, validators, resolvers and response templates.	100% of deterministic unit suite passes.
Retrieval / model evaluation	Top-1 intent, ranking, ambiguity handling, out-of-domain recall and false dispatch.	Intent exact match, slot exact match, escalation precision / recall, grammar validity.	Targets in sections 3.13 and 4.1.

Layer	Sam coverage	Ida coverage	Pass condition
Source / adapter	Index freshness, permission scope, source availability, handler timeouts, result provenance.	Mock OSAdapter and real set-then-read round trips for Act intents.	No unauthorised reads; every Act is verified.
Latency / memory	Cold and warm query p50/p95/p99, encoder load, index sizes, RSS, cache churn and 4 GB system pressure.	Per-stage wake, ASR, Tier 0, Tier 1, executor and first TTS timing; cold/warm and sustained load.	Meet measured budgets without unbounded memory growth.
Safety	Path traversal, denied roots, permission changes, untrusted document text, invalid action identifiers.	Out-of-scope paths, injected instructions, voice-only confirmations, destructive operations and undo.	Zero policy violations.
Privacy	No network egress; no query-history store; cache isolation; no sensitive content in cache.	Canary strings, consent revoke and outbound traffic capture.	Zero Sam egress; zero unconsented private data sent by Ida.
Fault injection	Kill indexer/encoder, corrupt registry, stale index, full disk, missing source.	Kill model / D-Bus, disconnect network, cloud failure, full disk, stale index.	Documented fallback and honest user-visible failure.
UX / daily use	Natural phrasing, speed perception, result relevance and escalation comprehension.	Task success, misunderstandings, perceived responsiveness and action confidence.	Sam: high trust and immediacy; Ida: at least 90% task success.

6.4 Sam evaluation-set composition

Category	Suggested share	Expected behaviour
Common clean local query	25%	Correct handler and result.
Natural paraphrase / synonym	25%	Same canonical intent.
Incomplete or colloquial wording	10%	Resolve only with sufficient context; otherwise clarify.
Ambiguous entity or intent	10%	Ask one concise clarification.
Stale / unavailable source	5%	Report source status; do not guess.
Out of domain	15%	Return typed handoff.
Adversarial / malformed input	10%	Reject, clamp or hand off; never execute arbitrary text.

6.5 Release channels

Sam and Ida should be versioned independently but tested together at integration points. A Sam registry update must not silently change Ida action behaviour. A compatible Essential OS release records the Sam registry version, handler bundle, embedding encoder (if present), index schema, Ida registry version, Tier 1 model version and evaluation results.

- Canary: frequent registry and handler changes, limited to the developer environment.
- Beta: broader domain coverage and low-resource validation with opt-in users or test machines.
- Stable: promoted only after frozen evaluation, safety, privacy, memory, latency and upgrade checks pass.

7. Risks, decisions and open questions

7.1 Sam risks

Risk	Mitigation	Fallback
Registry becomes dense but noisy	Canonical intents, deduplication, nearest-neighbour confusion checks and frozen evaluation.	Prefer fewer precise intents; return no-match rather than dispatching vaguely.
Embeddings add latency or memory cost	Lexical fast path first; load encoder on demand; benchmark actual model and vector storage.	Ship lexical-only mode.
Indexing harms system responsiveness	Incremental indexing, throttling, pause/resume and bounded background work.	Reduce domains or rebuild on demand.
Stale or incomplete sources	Track source version and freshness; refresh on change; expose stale state.	Report that a source is unavailable or stale.
Sam grows into an LLM-like assistant	Hard scope boundary, no generative runtime dependency, no memory and no reasoning loop.	Return typed handoff to caller.

Risk	Mitigation	Fallback
4 GB system under pressure	Hard RSS cap, lazy loading, disk-backed indexes and memory stress tests.	Unload encoder, use lexical path and defer nonessential indexing.
Sensitive local data exposed	Permission-aware indexes, deny lists, no secrets, cache isolation and no network egress.	Disable affected source and rebuild index.

7.2 Ida risks

Risk	Mitigation	Fallback
ASR too slow or inaccurate on CPU	Benchmark candidates and vocabulary biasing in Phase 0.	Smaller ASR model, push-to-talk and typed input.
Tier 1 cannot reach accuracy bar	Untrained baseline first, optional LoRA and calibrated thresholds.	Tier 0 plus embedding classifier and slot tagger; more escalation.
RAM pressure from resident models	Measure Phase 0 budget and load Tier 2 local on demand.	Smaller models and unload local mid after idle.
KDE / Wayland gaps	Verify each D-Bus / CLI mechanism on target Plasma.	Mark unsupported and explain clearly.
Wake-word false triggers	Conservative threshold and keypress confirmation for destructive actions.	Push-to-talk hotkey.
Privacy gate bypass	Resolver-assigned data classes, default-deny, canary tests.	Local-only mode.
Hermes cannot switch model per call	Early Phase 0 spike.	Thin proxy in front of Hermes.
Provider pricing or API changes	OpenAI-compatible abstraction and no provider lock-in.	Swap base URL and model name.
Solo developer bandwidth / scope creep	Each phase delivers a usable system; intent growth rule and measured gates.	Stop safely at a phase exit gate.

7.3 Open questions to resolve during implementation

- What are the exact 4 GB minimum reference-machine specifications and the baseline RAM/CPU usage of Essential OS without Sam?
- Which local data domains are enabled by default, and which require explicit user opt-in (for example, full-content document indexing)?
- Which dedicated embedding encoder, if any, gives the best quality/latency/RAM trade-off on CPU-only hardware?
- Which source owns each shared catalogue, and what event/version mechanism invalidates Sam results?
- What local IPC mechanism and versioned API should Sam expose to the global search UI and Ida?
- Which result actions may Sam offer directly (open file, launch app, open settings page), and which must always be routed through Ida or a separate OS permission service?
- What is the chosen wake phrase, and does it require a custom wake-word model?
- Which Tier 1 and local-mid model licences are acceptable?
- Should the keypress confirmation default for destructive actions be relaxed for any individual Ida intent after real-world testing?
- Which languages beyond English matter, and when?

Appendix A. Sam registry entry example

Illustrative schema only. The final schema should be kept in a machine-validated format and versioned with handlers, embeddings and evaluation cases.

```
{
 "id": "system.memory.top_consumers",
 "version": 1,
 "domain": "system.live_state",
 "canonical_intent": "list_memory_consumers",
 "description": "List processes using the most resident memory",
 "phrases": [
  "what is using my RAM",
  "what is eating my memory",
  "show the biggest memory users",
  "which apps are using the most RAM"
 ],
 "keywords": ["ram", "memory", "process", "usage", "consuming"],
 "embedding_text": [
  "Find applications and processes with the highest memory use"
 ],
 "slots": {
  "limit": {"type": "integer", "default": 5, "min": 1, "max": 20}
 },
 "source": "procfs.process_snapshot",
 "handler": "system.list_memory_consumers",
 "permissions": {"scope": "read_system_metrics"},
 "response_schema": "process_list.v1",
 "response_template": "Top {count} memory consumers",
 "cache_policy": {
  "mode": "short_ttl",
  "ttl_ms": 1500,
  "invalidate_on": ["process_table_changed"]
 },
 "fallback": {
  "on_source_error": "source_unavailable",
  "on_low_confidence": "no_confident_match"
 },
 "eval_cases": [
  "what is using my RAM",
  "show biggest memory users",
  "tell me the latest election result"
 ]
}
```

The handler is a fixed, reviewed implementation that reads the approved process-information source, applies a bounded result limit and returns structured metrics. It does not execute the query text or generate its own explanation.

Appendix B. Sam-to-caller contract example

B.1 Request


```
{
 "query": "why is my laptop getting hot",
 "scope": ["hardware", "system.live_state", "applications"],
 "caller": "essential_os_search",
 "capabilities": ["read_system_metrics", "read_process_metadata"],
 "locale": "en"
}
```


B.2 Successful response


```
{
 "status": "ok",
 "intent": "system.thermal_diagnostic_snapshot",
 "source_freshness": "live",
 "generated_at": "2026-11-03T00:00:00+03:00",
 "results": [
  {"type": "metric", "label": "CPU temperature", "value": 87, "unit": "°C"},
  {"type": "process", "label": "Top CPU consumer", "value": "example-process", "cpu_percent": 42.1}
 ],
 "actions": [
  {"id": "open_system_monitor", "label": "Open System Monitor"}
 ],
 "cache": {"eligible": false, "reason": "live_thermal_data"}
}
```


B.3 Handoff response


```
{
 "status": "handoff",
 "reason": "out_of_domain",
 "matched_domain": null,
 "message_key": "outside_local_computer_scope",
 "caller_action": "select_external_handler"
}
```


The caller may route the handoff to web search, a local model or a cloud model according to user settings and privacy policy. Sam does not select a provider, transmit the query or initiate the handoff itself.

B.4 Contract invariants

- The response is structured and versioned; no generated prose is required.
- Every result names its source or source class and indicates whether data is live, cached or indexed.
- A low-confidence match is not returned as a confident result.
- Sam never returns arbitrary shell commands or unrestricted executable text.
- The caller revalidates any action before executing it.
- No local content is sent externally as part of the Sam contract.

END OF SPECIFICATION